You can Prove it!

A Practical Tutorial in Writing Proofs

Elias Castegren & Ellen Arvidsson

September 5, 2025

This document was developed for the course Semantics of Programming Languages at Uppsala University. Please send any feedback to `elias.castegren@it.uu.se`.

Contents

Motivation 1

"What do you Mean by That?" 2

Exercises 5

"Oh yeah? Prove it!" 5

Implication 6

Equivalence 7

Conjunctions and Disjunctions 7

Quantifiers 8

Proof by Induction 9

Proof by Contradiction 12

Exercises 13

Suggested Solutions 14

"What do you Mean by That?" 14

"Oh Yeah? Prove it!" 14

Motivation

The purpose of this document is to refresh your knowledge about mathematical reasoning; in particular how to write *proofs*. The point of a proof is to present arguments that convince someone (possibly yourself!) that something is true. Proofs can have varying levels of formality, from completely informal text[1] to fully formal reasoning from definitions and axioms[2], and anything in between. How formal you should be depends on the context and who you are trying to convince; if you are reasoning with a friend or yourself you can probably be quite informal, but if you are designing a railway scheduling system being very formal, so formal that the proof can be checked by a computer,[3] can be a way of being very sure that two trains never meet on the same track. In this document we try to be pragmatic. We will write out and motivate each step of a proof, but not require reasoning from axioms every time.

[1] "Clearly $2 + 1 = 3$".

[2] "Let natural numbers be either 0 or the successor $S\ n$ of another natural number n. Let $0 + m = m$ and $(S\ n) + m = S\ (n + m)$. By the definition of 2 and 1, we have $2 + 1 = (S\ (S\ 0)) + (S\ 0)$. By the definition of $+$, we have $(S\ (S\ 0)) + (S\ 0) = S\ ((S\ 0) + (S\ 0)) = S\ (S\ (0 + (S\ 0))) = S\ (S\ (S\ 0)) = 3$ (by the definition of 3)".

[3] There is special kind of software called *proof assistants* which checks every step you make in a proof, ensuring that you only take correct steps.

This document should not be seen as a beginner's text book in logic or even proofs – the text assumes that you have a basic understanding of both logic and proofs. The first goal of this document is to have you brush up on this understanding and remind you about concepts and terminology. A large part of doing proofs is quite mechanical, but learning how to operate the machinery requires practice. The second goal is therefore to provide you with a set of practical tools that you can use when doing your own proofs. If you are taking the course Semantics of Programming Languages, this document also sets the expectations of the level of formality in seminars and assignments.

The rest of this tutorial has two main sections. The first section covers how to translate informal statements into formal ones – that is, how to translate natural language into mathematical symbols that clearly and unambiguously express what you mean. The second section covers techniques for proofs, sorted by the kinds of constructs you might come across and how to handle them. Each section comes with a couple of exercises. Suggested solutions are provided at the end of this document.

"What do you Mean by That?"

Before you can even start to prove anything, you should understand what every part of the thing you are proving means. In other words, you need to know how to *state* what it is you want to prove in a clear way. We do this by translating informal statements written in text to formal statements written using mathematical symbols. One of the points of this is to get rid of the ambiguity that is prevalent in natural languages, for example in how we use conjunctions like "or" when we speak. If you are asking someone if they would like tea or coffee, you are implicitly expecting them to pick one of them, and not both.[4] As you know, in logic there is a distinction between "inclusive or" and "exclusive or", each with their own (clearly defined) semantics.

[4] While it is logically correct to answer "yes" to the question "Would you like tea or coffee?", it is also quite obnoxious. Don't be that person.

Once you get used to doing proofs you can afford to be less formal in your formulations, but in order to learn the mechanics of mathematical reasoning it helps to be explicit. Sometimes an exercise or assignment will give you a formal statement to begin with, but most times you will not be so lucky.

When dealing with proofs, we typically call a statement that can be true or false (but not both) a *proposition*.[5] Here are three examples of propositions:

[5] Often we will use "derivable" instead of "true", since we might be looking at what we can derive with a given set of rules rather than be concerned with the philosophical concept of truth.

- "Three is less than two"
- "There is at least one natural number between five and ten"
- "Bananas are yellow"

Note that even though the first example is clearly not true, it is still a proposition! In order to write the third statement down with mathematical symbols, we need a way to express the concepts of "something being a banana" and "something being yellow". These are examples of *predicates*. Predicates can be thought of as functions whose result is a proposition; we can write $banana(x)$ or $yellow(x)$ for the propositions that x (whatever it is) is a banana or yellow respectively. Note that $yellow$ is a predicate, while $yellow(x)$ is a proposition (assuming x is defined).

With these predicates, we can write the propositions formally as follows:

- $3 < 2$
- $\exists n \in \mathbb{N}.\ 5 < n \ \wedge\ n < 10$
- $\forall b. banana(b) \implies yellow(b)$

We are not claiming that these statements are necessarily *better* than the informal ones; one might even argue that the informal ones are easier to understand. However, we have gotten rid of some ambiguity from the informal statements. For example, when we say that "Bananas are yellow", do we mean that *all* bananas are yellow or that *there are* bananas which are yellow.[6] When formalising the proposition, we get rid of this ambiguity:

- "All bananas are yellow" – $\forall b. banana(b) \implies yellow(b)$
- "Some bananas are yellow" – $\exists b. banana(b) \wedge yellow(b)$

Note also that have not *defined* how to identify a banana or that an object is yellow, which may or may not be important for what you want to do with these predicates. The important step for now is that we have put a name on these properties.[7]

Another thing to note is that we used (for example) the symbols $<$, 2 and 3 without introducing them. This is because they already have meanings that a reader can be expected to understand. Indeed, the symbol $<$ is also a predicate, but one that takes two arguments (note how $3 < 2$ is a proposition). A predicate that takes two or more arguments is often called a *relation*.

Example 1 There is a Swedish children's song (to the tune of "Oh Tannenbaum") called "En busschaufför" whose lyrics let us know that "A bus driver is a man with a happy mood".[8] In order to formalise this proposition, we start by introducing three predicates:

- $busDriver(p)$ – p is a bus driver
- $male(p)$ – p identifies as male

Glossary

- A *proposition* is a statement that is either true or false.
- A *predicate* is a function whose result is a proposition.
- A *relation* is a predicate which takes two or more arguments.

[6] This distinction turns out to be important: only the latter interpretation is true since there are bananas which are red!

[7] There is a *symbolic representation* of the properties

[8] Swedish children learn the logical concept of the *contrapositive* from an early age. The song continues to state that "if he does not have a happy mood, then he is no bus driver": $\forall p. \neg happy(p) \implies \neg busDriver(p)$

- $happy(p)$ – p has a happy mood

The proposition of the song can now be formalised as "for any person p, if p is a bus driver, then p is male and p has a happy mood":

$$\forall p.busDriver(p) \implies male(p) \wedge happy(p)$$

Again, we can learn from this example is that it is perfectly possible to formalise propositions which are not true: most of us will have met bus drivers who do not identify as male or who we would not classify as happy.

□

We will end this section with an example that is more connected to computer science than the colour of bananas and the mood of bus drivers.

Example 2 A list is either empty or consists of one element followed by the rest of the list. A list can be *filtered* given some predicate P, such that elements that do not satisfy P are discarded. One important property of filtering is that if an element for which P holds is in a list, this element will also be in the list if it is filtered with P.[9]

If we assume the existence of a *filter* operation and a relation *member* that holds if a given integer is in a given list, we can formalise the this property as a proposition:

$$\forall P, x, l.\ P(x) \wedge member(x, l) \implies member(x, filter(l, P))$$

We will go ahead and formalise this with enough details so that we can actually prove that what we have said holds (although the proof will be given in a later section).

We start by defining the syntax of lists, keeping the elements abstract.[10] We write this in Backus-Naur form, use ϵ to mean "the empty list" and separate elements of the list with semicolons:

$$l ::= \epsilon \mid x; l$$

We then continue to define the operation *filter* which produces a filtered list given some predicate P. We write this as a function over l as defined above:

$$filter(l, P) = \begin{cases} \epsilon & \text{if } l = \epsilon \\ x; filter(l', P) & \text{if } l = x; l' \text{ and } P(x) \text{ holds} \\ filter(l', P) & \text{if } l = x; l' \text{ and } P(x) \text{ does not hold} \end{cases}$$

What this is saying is that whenever we have a term $filter(l, P)$ we can replace it by (since it is equal to) whatever line on the right-hand side for which the side-conditions hold.[11]

Finally, we define the derivation rules for when an element is a member of a list; it is either the first element of the list, or it is a member of the rest of the list:

$$\frac{}{member(x, x; l)} \qquad \frac{member(x, l)}{member(x, y; l)}$$

□

[9] This is a property we want, but it is not enough. Try to think about a definition of filtering that is wrong but which still satisfies this property.

[10] We could for example declare some set of possible list elements $\mathbb{E}$ and say that x is some element of that set: $x \in \mathbb{E}$.

[11] We defined addition of natural numbers as a function in footnote 2. We could write it equivalently using side-conditions as follows:

$$n + m = \begin{cases} m & \text{if } n = 0 \\ S\ (n' + m) & \text{if } n = S\ n' \end{cases}$$

Exercises

1. The TLC song "No Scrubs" from the 1999 album FanMail makes the following statement: "A scrub is a guy that can't get no love from me". Formalise this statement, assuming that it applies to all the members of TLC. Start by introducing any predicates, sets or functions you might need.[12]

2. The offside rule in soccer states that a player is in "offside position" if they are

   (a) in the opponents' half of the pitch;

   (b) closer to the opponents' goal than the ball is; and

   (c) closer to the opponents' goal than the second-closest opponent (the opponent closest to the goal is typically the goal keeper)

   Define the proposition "player p is in offside position". For simplicity you may assume that the pitch is a one-dimensional line and that both the ball and the players are distinct points on this line. Write your proposition as an equivalence, *i.e.* $\forall p.\mathit{offside}(p) \iff \ldots$, where $\ldots$ is your definition. You may want to introduce helper functions for getting different positions on the pitch or for getting sets of players.[13]

[12] "Can't get no" sounds like a double negative, but should be interpreted as "Can't get any".

[13] **Hint:** Set comprehensions may also come in handy. For example, the set of all even natural numbers can be written as $\{n \mid n \in \mathbb{N} \wedge n \; mod \; 2 = 0\}$

"Oh yeah? Prove it!"

How does one ensure that only correct steps are taken in a proof? This is varies depending on how formal you want to be in your proof. In the end, the point of a proof is convincing someone that a proposition holds. Sometimes the right level of formality is just using natural language, and sometimes the right level is to derive every step from logical axioms. Here, we will take a middle route where we write all propositions formally, but write our reasoning using natural language. We will cover how to work with different logical constructs that you might come across while working on a proof. We also cover two proof techniques: proof by induction and proof by contradiction.

All proofs have a *goal*, the thing you are trying to prove holds. You also have a set of *assumptions*, things that you are assuming holds already.[14] Both the goal and the assumptions are propositions as seen in the previous section. The process of conducting a proof is using logical steps starting from the assumptions and hopefully arriving at the goal.

The mechanical steps involved in handling the different logical constructs covered here are different depending on whether the construct

[14] Sometimes assumptions are called *hypotheses*. Note that we use the same terminology even for things that are obviously true (like $1 = 1$), even though it seems to imply that we might not be sure.

appears as a goal or in an assumption. Note also that there is no algorithm for how to prove anything – this section gives you tools that you can apply when writing proofs.

Implication

One of the most basic logical constructs is the implication $A \implies B$. It states that if we know A, then we can also know B.[15] There are two important things to note. Firstly, there is nothing that says that we *have to* use A when proving B, just that we are allowed to. Secondly, if we happen to know that A does *not* hold, this does not say anything about whether B holds or not.

[15] An implication $A \implies B$ is like a function whose argument is (a proof of) A and whose result is (a proof of) B.

In the Goal If our goal is an implication $A \implies B$, we are asked to show that we can derive B starting from A (and whatever other assumptions we may have). We do this by adding A to our set of assumptions and proceeding to show B.

As an Assumption If we have an implication $A \implies B$ as an assumption (or as a previously proved proposition), we may derive B if we already know A.

Examples

- For any n, show $n < 5 \implies n + 2 < 7$.
  Proof
  1. We start by assuming $n < 5$. We need to show $n + 2 < 7$.
  2. By adding two on both sides of $n < 5$ we get $n + 2 < 5 + 2$.
  3. By basic arithmetics, $5 + 2 = 7$, giving $n + 2 < 7$. $\square$
- For any n, show $n < 5 \implies 5 < 7$.
  Proof
  1. We start by assuming $n < 5$. We need to show $5 < 7$, which holds trivially.[16] $\square$
- Assuming $A \implies B$ and $B \implies C$, show $A \implies C$.
  Proof
  1. We start by assuming A. We need to show C.
  2. By the first assumption,[17] having A gives us B.
  3. By the second assumption, having B gives us C. $\square$

[16] Note that we didn't need to use the assumption here, but we still "peel it off" in order to reach the right-hand side.

[17] With only two assumptions, we can get away with just referring to them as "the first" and "the second". For larger proofs it is a good idea to give names to your different assumptions.

A special case of proofs involving implications occur when the left-hand side of an implication does not hold. Remember that for $A \implies B$ we are only asked to show B *if* A holds. If A does not hold, we don't have to do anything; we say that the proposition *holds vacuously*. Similarly, if one of our assumptions turns out to be false, we

can conclude the proof, since we have ended up in a situation that cannot happen.[18] This also relates to proofs by contradiction, which is covered later.

[18] This is known as the *principle of explosion*, or by the Latin phrase *Ex falso quodlibet*, "from falsehood [follows] anything".

Equivalence

An equivalence $A \iff B$ says that A is true if *and only if* B is true. It is equivalent (pun intended) to having two implications, $A \implies B$ and $B \implies A$.

In the Goal If our goal is an equivalence $A \iff B$, we need to show both directions of the equivalence. In other words, we need to show both $A \implies B$ and $B \implies A$.

As an Assumption If we have an equivalence $A \iff B$ as an assumption, we may use this assumption to show B if we already have A, or to show A if we already have B.[19]

[19] Again, it is as if we had two assumptions, one for each direction of the equivalence

Example

- For any natural number n, show that $n < 10 \iff n^2 < 100$.
  Proof
  1. We start by showing the first direction $n < 10 \implies n^2 < 100$:
     (a) We start by assuming $n < 10$. We need to show $n^2 < 100$.
     (b) By squaring both sides of $n < 10$, we get $n^2 < 10^2$.
     (c) By basic arithmetics, $10^2 = 100$, giving $n^2 < 100$.
  2. Next we show the second direction $n^2 < 100 \implies n < 10$:
     (a) We start by assuming $n^2 < 100$. We need to show $n < 10$.
     (b) By taking the square root of both sides of $n^2 < 100$, we get $\sqrt{n^2} < \sqrt{100}$.
     (c) By basic algebra, $\sqrt{n^2} = n$,
     (d) By basic arithmetics, $\sqrt{100} = 10$, giving $n < 10$.

  $\square$

Conjunctions and Disjunctions

Conjunction (written $\wedge$) and disjunction ($\vee$) are the fancy names of logical *and* and logical *or*. A conjunction $A \wedge B$ is true if both A and B are true. A disjunction $A \vee B$ is true if at least one of A and B are true.

Connectives
Implication, equivalence, conjunctions and disjunctions are all examples of logical *connectives*. A connective is a predicate whose arguments are one or more propositions. Another connective that you are likely to come across is negation $\neg A$.

In the Goal If our goal is a conjunction $A \wedge B$, we need to show both A and B. We can do this by splitting our proof into two subproofs, just as we did for the equivalence proof above. If our goal is a

disjunction $A \vee B$, we need to show either A or B but it is enough to show one of them.

As an Assumption If we have a conjunction $A \wedge B$ as an assumption, we get to assume both A and B. If we have a disjunction $A \vee B$ as an assumption, since either A or B must be true, we can split our proof into two sub-proofs, one where we assume only A and one where we assume only B.[20]

[20] Note that we are not assuming that if A is true, then B is false. It is possible that they are both true at the same time.

Examples

- Show commutativity of conjunction, *i.e.* $A \wedge B \implies B \wedge A$.
  Proof
  1. We start by assuming $A \wedge B$. We need to show $B \wedge A$
  2. Our assumption lets us assume A and B.
  3. We can show $B \wedge A$ by showing B and A separately, both of which hold by assumption. $\square$
- Show commutativity of disjunction, *i.e.* $A \vee B \implies B \vee A$.
  Proof
  1. We start by assuming $A \vee B$. We need to show $B \vee A$.
  2. We proceed by cases over the assumption $A \vee B$:
     (a) If A holds, then that is enough to show $B \vee A$.
     (b) If B holds, then that is enough to show $B \vee A$.

  $\square$

Quantifiers

Quantifiers are used to give names to things for some scope. The two most common quantifiers in logic are "for all" ($\forall$), used to introduce a name that represents all instances of something, and "exists" ($\exists$), used to introduce a name that represents a particular but unspecified instance of something.

In the Goal If our goal has the form $\forall x.P(x)$, where $P(x)$ can be any proposition mentioning x,[21] we can assume that we have *some* abstract value v and proceed by showing that $P(v)$ holds. Note that we can pick any name for v (assuming we update P with the new name). Often we will reuse the name of the quantified variable (x above), but renaming may be necessary in order to avoid name clashes. If our goal has the form $\exists x.P(x)$, we need to provide some concrete value v such that $P(v)$ holds (or derive a contradiction from our assumptions).

[21] Or not. There is no rule that requires x to be mentioned, but if it isn't mentioned there is no point in the quantifier being there.

As an Assumption If we have an assumption $\forall x.P(x)$, we are allowed to assume that $P(v)$ holds for *any* concrete value v.[22] If we have an assumption $\exists x.P(x)$, we are allowed to assume that we have some abstract value v such that $P(v)$ holds.

[22] In practice, there will almost always be some kind of precondition on x, for example that it is natural number: $\forall x \in \mathbb{N}.\ P(x)$

Examples

- Show $\forall n \in \mathbb{N}.0 < n \implies \exists m \in \mathbb{N}.m < n$.
  Proof
  1. We start by assuming that we have some natural number n such that $0 < n$. We need to show $\exists m \in \mathbb{N}.m < n$.
  2. We pick 0 as a concrete value for m. We need to show $0 < n$ which holds by assumption. □
- Show $\forall n \in \mathbb{N}.(\exists m \in \mathbb{N}.n = m + 1) \implies n \neq 0$.
  Proof
  1. We start by assuming that we have some natural number n and that $\exists m \in \mathbb{N}.n = m + 1$. We need to show $n \neq 0$.
  2. We assume that we have some natural number m such that $n = m + 1$.
  3. We need to show $m + 1 \neq 0$, which holds trivially since m cannot be smaller than 0, the smallest natural number. □

Duality
Note how the two quantifiers behave sort of like opposites: we are allowed to assume the existence of a value when we have a $\forall$ in the goal or when we have an $\exists$ in our assumptions, but we have to provide a value when we have a $\forall$ in our assumptions or an $\exists$ in our goal. This is an example of a *duality*, a deep mathematical connection that often pops up in formal logic.

Proof by Induction

Some kinds of values can only take on a finite number of forms. This allows us to do proofs by enumeration. For example, if we wanted to show that $A \implies A \vee B$ we can just write out the truth table for all possible values of A and B and see that the result is always true:[23]

A	B	$A \implies A \vee B$
F	F	$F \implies F \vee F \equiv T$
F	T	$F \implies F \vee T \equiv T$
T	F	$T \implies T \vee F \equiv T$
T	T	$T \implies T \vee T \equiv T$

[23] In propositional logic, the *theorem of completeness* states that if all enumerations of truth values result in a proposition being true, then there is also a proof of that proposition.

However, a lot of the time the things we reason about can be arbitrarily large.[24] For example, if we want to prove something about natural numbers we will not be able to list all of them. The standard technique for dealing with this is doing a proof by induction.

[24] Sometimes even infinite, but we try to avoid that whenever possible since infinities are weird...

If we want to prove some property P about natural numbers, a proof by induction is done in two steps: the *base case*, where we show that P holds for the number 0, and the *inductive case* where we show that P holds for $k + 1$, assuming P holds for some number k. The intuition is that the base case gives us $P(0)$, and then the inductive case first gives us $P(1)$ (for $k = 0$), and then $P(2)$ (for $k = 1$), and so on up to any number.[25] The assumption that P holds for some k in

[25] You can think of an inductive proof as defining a *procedure* for how to prove the property for any given number.

the inductive case is known as the *induction hypothesis*.

Example 3 Show that multiplication distributes over addition for natural numbers: $\forall n, a, b.\ n \cdot (a + b) = n \cdot a + n \cdot b$.

Proof
We assume that we have some natural numbers n, a and b. We need to show $n \cdot (a + b) = n \cdot a + n \cdot b$. We proceed by induction over n.[26]

- **Base Case**: $n = 0$
  Need to show $0 \cdot (a + b) = 0 \cdot a + 0 \cdot b$
  1. By arithmetics, we have $0 \cdot (a + b) = 0$.
  2. Similarly, we have $0 = 0 \cdot a + 0 \cdot b$, thus $0 \cdot (a + b) = 0 \cdot a + 0 \cdot b$.
- **Inductive Case**: $n = k + 1$
  Need to show $(k + 1) \cdot (a + b) = (k + 1) \cdot a + (k + 1) \cdot b$
  Induction Hypothesis: $k \cdot (a + b) = k \cdot a + k \cdot b$
  1. By the definition of multiplication, we have $(k + 1) \cdot (a + b) = k \cdot (a + b) + (a + b)$.
  2. By the induction hypothesis we have $k \cdot (a + b) + (a + b) = k \cdot a + k \cdot b + (a + b)$.
  3. By the definition of multiplication, we have $k \cdot a + k \cdot b + (a + b) = (k + 1) \cdot a + (k + 1) \cdot b$, thus $(k + 1) \cdot (a + b) = (k + 1) \cdot a + (k + 1) \cdot b$.

□

[26] It is always important to state which natural number you are doing induction over as there is often more than one number involved.

Note how the goal changes in the different cases. To make this explicit above we repeat "need to show" in the different cases so there's less to remember implicitly. We make clear when we start our induction proof and which case we are on. For each step of the proof, we do something that brings us closer to our goal and write why this step is motivated. Ideally, the last step of every (sub) proof contains the exact symbols listed as "need to show" at the start.

You have probably seen induction proofs used for natural numbers before, but the technique generalises to anything that has an *inductive structure*, that is anything defined in terms of smaller versions of itself. This includes trees, lists, programs, *etc.* For example, recall the definition of lists from Example 2:

$$l ::= \epsilon \mid x; l$$

We can prove some property P of lists by using *structural induction*. We start by showing the base case, that P holds for the empty list ϵ, and then we show the inductive case, that P holds for some arbitrary but non-empty list $x; l$, assuming that P holds for the sublist l. The intuition is the same as for induction over numbers: the base case gives us $P(\epsilon)$, and then the inductive case gives us $P(x; \epsilon)$ (for any x and for $l = \epsilon$), and then $P(y; x; \epsilon)$ (for any y and for $l = x; \epsilon$), and so on up to a list of arbitrary length.[27]

[27] The similarities become even more apparent when you consider Peano's definition of natural numbers n: $n ::= 0 \mid S\ n$. A natural number is either zero, or the successor of another natural number.

Example 4 In Example 2 we formalised a proposition relating to filtering of lists that we will now prove. For convenience we list the definition of *filter* and the derivation rules of *member* again:

$$\mathit{filter}(l, P) = \begin{cases} \epsilon & \text{if } l = \epsilon \\ x; \mathit{filter}(l', P) & \text{if } l = x; l' \text{ and } P(x) \text{ holds} \\ \mathit{filter}(l', P) & \text{if } l = x; l' \text{ and } P(x) \text{ does not hold} \end{cases}$$

$$\frac{}{\mathit{member}(x, x; l)} \qquad \frac{\mathit{member}(x, l)}{\mathit{member}(x, y; l)}$$

The proposition we want to prove is the following:

$$\forall x, l.\ P(x) \ \wedge\ \mathit{member}(x, l) \implies \mathit{member}(x, \mathit{filter}(l, P))$$

We start by assuming we have some element x and list l such that $P(x)$ and $\mathit{member}(x, l)$.[28] We need to show $\mathit{member}(x, \mathit{filter}(l, P))$. We proceed by structural induction over l.

[28] Here we are introducing quantified variables x and l, assuming the left-hand side of the implication and splitting the conjunction $P(x) \ \wedge\ \mathit{member}(x, l)$ in a single step.

- **Base Case**: $l = \epsilon$
  Need to show $\mathit{member}(x, \mathit{filter}(\epsilon, P))$
  Assumption 1: $\mathit{member}(x, \epsilon)$
  Assumption 2: $P(x)$
  1. By *Assumption 1*, we have $\mathit{member}(x, \epsilon)$. But this is impossible, since both derivation rules of *member* requires a non-empty list. The base case thus holds vacuously.
- **Inductive Case**: $l = y; l'$ [29]
  Need to show $\mathit{member}(x, \mathit{filter}(y; l', P))$
  Induction Hypothesis:[30] $\mathit{member}(x, l') \implies \mathit{member}(x, \mathit{filter}(l', P))$
  Assumption 1: $\mathit{member}(x, y; l')$
  Assumption 2: $P(x)$
  There are two rules that can be used for deriving $\mathit{member}(x, y; l')$. We proceed by cases over *Assumption 1*:
  - **Case 1**: $x = y$.
    Need to show $\mathit{member}(x, \mathit{filter}(x; l', P))$.
    1. By the definition of *filter* (case 2, since we have $P(x)$ from *Assumption 2*) we have $\mathit{filter}(x; l', P) = x; \mathit{filter}(l', P)$.
    2. We have $\mathit{member}(x, x; \mathit{filter}(l', P))$ by the first derivation rule of *member*.
  - **Case 2**: $\mathit{member}(x; l')$ (*Assumption 3*).
    Need to show $\mathit{member}(x, \mathit{filter}(y; l', P))$.
    1. By the definition of *filter* there are two possible values of $\mathit{filter}(y; l', P)$ depending on whether y is removed or not.
    2. If y is removed we have $\mathit{member}(x, \mathit{filter}(l', P))$ by the induction hypothesis with *Assumption 3*.
    3. If y is not removed, we have $\mathit{member}(x, y; \mathit{filter}(l', P))$ by the second derivation rule of *member*, getting the premise from the induction hypothesis with *Assumption 3*.

[29] We pick the name y since x is already in use in our assumptions.

[30] Note that the induction hypothesis turns into an implication when we have assumptions that mention the variable we are doing induction over. This is because the whole thing we are trying to prove is that our assumptions imply our goal. If we just used $\mathit{member}(x, \mathit{filter}(l', P))$ as the induction hypothesis, our proof would not be valid.

□

Most parts of a proof by induction can be done mechanically without thinking. The creative parts is choosing which variable to do induction over and figuring out which steps to take in order to get to the thing you have to prove. As long as we have a clear structure, filling in the cases, assumptions, the induction hypothesis and what the current goal is should be automatic.[31]

[31] That is not to say that it is not something that requires practice!

Proof by Contradiction

Another way (some would say *the* other way) to prove that something holds for all values of some infinite set (like the natural numbers) is to show that it cannot *not* be the case. We typically accept that all propositions are either true or false,[32] and if we can show that a proposition is not false then it must be true. This is known as a proof by contradiction.

[32] This is known as the *law of excluded middle*; there is no truth value between true and false.

A proof by contradiction of some property P starts out by assuming $\neg P$, *i.e.* that P does not hold, and then tries to derive a contradiction.[33] If we can derive a contradiction, our assumption of $\neg P$ must have been wrong, thus it *must* be the case that P holds. It is not always obvious when a proof by contradiction is the right choice of method, but it is something that is useful to have in one's tool box.

[33] A contradiction is when we can show that an assumption (which we have assumed is true) is actually false.

Example 5 Show that there exists two irrational numbers a and b such that a^b is rational:

$$\exists a, b \notin \mathbb{Q}.a^b \in \mathbb{Q}$$

Assume for contradiction that such a pair of irrational numbers does *not* exist, *i.e.* that a^b is always irrational for irrational a and b.

1. We know from a classic result that $\sqrt{2}$ is irrational, so by our assumption $\sqrt{2}^{\sqrt{2}}$ is also irrational.[34]
2. This should mean that $\left(\sqrt{2}^{\sqrt{2}}\right)^{\sqrt{2}}$ is irrational, but $\left(\sqrt{2}^{\sqrt{2}}\right)^{\sqrt{2}} = \left(\sqrt{2}^{\sqrt{2}\cdot\sqrt{2}}\right) = \sqrt{2}^2 = 2$ which is rational! This contradicts our assumption that a^b is always irrational for irrational a and b.

□

[34] We do not know whether $\sqrt{2}^{\sqrt{2}}$ is irrational or not, but we are working under the assumption that a^b is always irrational for irrational a and b. It turns out that $\sqrt{2}^{\sqrt{2}}$ is indeed irrational, but our proof would be valid regardless.

Note that while we have showed that there must exist two irrational numbers a and b such that a^b is rational, we have not showed what these numbers are. We have only showed that they cannot *not* exist! This is typical for proofs by contradiction.[35]

[35] We say that a proof by contradiction is not *constructive*.

A subtle detail is that a proof can derive a contradiction without being a proof by contradiction![36] For example, if you are asked to prove $\neg P(x)$, *i.e.* that $P(x)$ does *not* hold, the way you go about it is to assume $P(x)$ and then show that this leads to a contradiction. This is a direct proof of a negative proposition ($\neg P(x)$). A proof by

[36] If you are on a computer, you can read a rant about it here: `https://existentialtype.wordpress.com/2017/03/04/a-proof-by-contradiction-is-not-a-proof-that-derives-a-contradiction`. This is also where I got the previous example from.

contradiction would be a proof of some positive proposition ($P(x)$) that proceeds by showing that its negation ($\neg P(x)$) does *not* hold.[37]

[37] Don't worry if this distinction does not make sense to you! It will most likely not affect you in any meaningful way.

Example 6 A classic proof that derives a contradiction is showing that there is an infinite number of prime numbers. Recall that a number is prime if it is only divisible by itself and 1, and that all numbers which are not prime have factors which are prime. As a technicality, also recall that if n is a factor of both a and b, then it is also a factor of the difference of a and b.[38]

[38] Assume a is the larger of the two numbers a and b. If n divides a and b, then $\frac{a}{n}$ and $\frac{b}{n}$ are integers. We can derive the equality $a - b = n\frac{a}{n} - n\frac{b}{n} = n(\frac{a}{n} - \frac{b}{n})$, which is clearly divisible by n.

To show that the number of primes is infinite, *i.e.* that is it not finite, we start by assuming that there is a finite number of primes and list all of them as $p_1, p_2, \ldots, p_n$.

1. Let $P = p_1 \cdot p_2 \cdots p_n$ be the product of all primes and consider the number $P + 1$. We know that $P + 1$ is not a prime number (we just listed all of them!), so $P + 1$ must have some prime factor p.
2. Since we just listed all of the primes, p must be one of $p_1, p_2, \ldots, p_n$, meaning it is also a prime factor of P.
3. Since p divides both P and $P + 1$, it must also divide $P + 1 - P = 1$. However, no prime number divides 1, so p cannot be one of $p_1, p_2, \ldots, p_n$. This contradicts our assumption that $p_1, p_2, \ldots, p_n$ contains all prime numbers. Therefore the prime numbers must be infinitely many.

□

Exercises

3. Show that conjunction distributes over disjunction: $\forall A\ B\ C.\ A \wedge (B \vee C) \iff (A \wedge B) \vee (A \wedge C)$. You could do this by building a truth table, but try to do it using a step-by-step proof.

4. Use induction to show that $n^3 + 11n$ is divisible by 6 for any natural number n.[39]

[39] **Hint**: Remember that a number n is divisible by 6 if and only if there exists some number k such that $n = 6k$.

5. The length of a list can be defined as follows:

$$\mathit{length}(l) = \begin{cases} 0 & \text{if } l = \epsilon \\ 1 + \mathit{length}(l') & \text{if } l = x; l' \end{cases}$$

The concatenation of two lists can be defined as follows:

$$\mathit{concat}(l_1, l_2) = \begin{cases} l_2 & \text{if } l_1 = \epsilon \\ x; \mathit{concat}(l_1', l_2) & \text{if } l_1 = x; l_1' \end{cases}$$

Use structural induction to show that the length of the concatenation of two lists equals the sum of their individual lengths.

6. In Example 4 we showed that a filtered list keeps all elements that satisfy the given predicate. Another expected property of filtering is that a filtered list only contains elements that satisfy the predicate. Formalise this statement as a proposition and prove that it holds.

7. Show that filtering the concatenation of two lists results in the concatenation of the filtered versions of the individual lists.

Suggested Solutions

Some suggested solutions are omitted before the seminar.

"What do you Mean by That?"

1. All three members of TLC share the opinion that anyone who is a scrub is a guy who cannot get any love from them. We introduce a set TLC of members of TLC, the predicates guy and $scrub$, and a function $love$ such that $love(a, b)$ returns a natural number representing the amount of love a is willing to give to b. We use the "for all" quantifier to introduce names p and q for different people:

$$\forall p.p \in TLC \implies (\forall q.scrub(q) \implies guy(q) \wedge love(p, q) = 0)$$

Note that we can put both quantifiers in the beginning and turn the proposition into a single implication:

$$\forall p, q.p \in TLC \wedge scrub(q) \implies guy(q) \wedge love(p, q) = 0$$

"Oh Yeah? Prove it!"

3. We want to show $\forall A\ B\ C.\ A \wedge (B \vee C) \iff (A \wedge B) \vee (A \wedge C)$. We start by assuming we have some propositions A, B and C and proceed to show $A \wedge (B \vee C) \iff (A \wedge B) \vee (A \wedge C)$. We proceed by cases over the two directions:

Case 1: $A \wedge (B \vee C) \implies (A \wedge B) \vee (A \wedge C)$
We start by assuming $A \wedge (B \vee C)$. We need to show $(A \wedge B) \vee (A \wedge C)$. From our assumption we get A and $B \vee C$. We proceed by cases over $B \vee C$:

(a) If B holds then we can show that $A \wedge B$ holds since both A and B hold.

(b) If C holds then we can show that $A \wedge C$ holds since both A and C hold.

Case 2: $(A \wedge B) \vee (A \wedge C) \implies A \wedge (B \vee C)$
We start by assuming $(A \wedge B) \vee (A \wedge C)$. We need to show $A \wedge (B \vee C)$. We proceed by cases over $(A \wedge B) \vee (A \wedge C)$:

(a) If $(A \wedge B)$ holds, then A and B hold. Now A holds by assumption and from B we can show $B \vee C$.

(b) If $(A \wedge C)$ holds, then A and C hold. Now A holds by assumption and from C we can show $B \vee C$.